JAVA PROGRAMMING

EXCEPTION HANDLING • Handwritten-style study notes

Topics: Errors & exceptions, hierarchy, checked/unchecked, try-catch-finally, throw/throws, propagation, custom exceptions, resources, assertions and best practices.

PROGRAM EXECUTION
↓
Problem occurs?
↙ ↘
No → continue normally Yes → create / throw a Throwable
↓
Matching handler?
↙ ↘
Yes: handle No: propagate / terminate

Learning goals

- *Understand why Java uses exception handling.*
- *Choose the right handling construct for each situation.*
- *Write readable, safe code that closes resources and preserves useful error information.*

Remember: An exception is an object that represents an unusual condition during program execution. Exception handling separates error-recovery code from normal program logic.

1. Error vs Exception and hierarchy

Java places errors and exceptions under the common superclass Throwable. Only objects of Throwable (or its subclasses) can be thrown with throw and caught with catch.

java.lang.Object
↓
java.lang.Throwable
↙ ↘
Error Exception
(serious JVM/system) ↙ ↘
RuntimeException Other exceptions
(unchecked) (usually checked)

Error

An Error indicates a serious problem generally outside the application's control. Applications normally should not try to recover from it.

- *OutOfMemoryError – JVM cannot allocate required memory.*
- *StackOverflowError – call stack is exhausted, often due to deep recursion.*
- *LinkageError – a class-loading or linkage problem.*

Exception

An Exception represents a condition an application may be able to handle, such as invalid input, a missing file or a failed network operation.

- *IOException – input/output operation failed.*
- *SQLException – database operation failed.*
- *RuntimeException – programming or logic problem such as a null reference or invalid index.*

Quick comparison

Error: serious environment/JVM issue; recovery is uncommon. Exception: application-level condition; can often be caught, reported or handled.

```
try {
    int answer = 10 / 0;
} catch (ArithmeticException ex) {
    System.out.println("Cannot divide by zero");
}
```

2. Checked vs unchecked exceptions

Checked exceptions are checked by the compiler. A method must catch them or declare them with throws. Unchecked exceptions are RuntimeException subclasses (and Error subclasses); the compiler does not require catch/declare.

Throwable
├── Error → unchecked
└── Exception
├── RuntimeException → unchecked
│ ├── NullPointerException
│ ├── ArithmeticException
│ └── ArrayIndexOutOfBoundsException
└── Other Exception subclasses → checked
├── IOException
├── SQLException
└── ClassNotFoundException

Examples and handling

```
// Checked: compiler requires handling or declaration
import java.io.*;

class ReadDemo {
    static void openFile() throws IOException {
        FileReader reader = new FileReader("data.txt");
    }
}

// Unchecked: no mandatory catch / throws
int[] values = {10, 20};
System.out.println(values[5]); // ArrayIndexOutOfBoundsException
```

When to use

• Use checked exceptions when the caller can reasonably take a meaningful recovery action.

• Use unchecked exceptions for programming mistakes, violated preconditions or situations the caller cannot usefully recover from.

• Do not catch an exception merely to silence the compiler.

3. try, catch and finally

try contains code that may fail. A matching catch handles the exception. finally runs after try/catch in normal control flow and is commonly used for cleanup.

```
try {
    int result = 20 / 0;
    System.out.println(result);
} catch (ArithmeticException ex) {
    System.out.println("Division is not possible");
} finally {
    System.out.println("This cleanup section runs");
}
```

Start
↓
Enter try block
↓
Exception occurs?
↙ ↘
No → skip catch Yes → find matching catch
↓ ↓
finally block ←———————— handler executes
↓
Continue after try-catch-finally

Important rules

• *A try block must be followed by at least one catch or a finally block.*

• *A catch handles only a compatible exception type.*

• *finally usually runs whether an exception occurs or not, including when a return executes in try/catch.*

• *Avoid return or throwing a new exception from finally: it can hide the original result or exception.*

• *Process-fatal termination (for example, JVM halt) may prevent finally from running.*

4. Multiple and nested catch

Multiple catch blocks

Use multiple catch blocks when different exception types need different responses. Put more specific exception types before more general parent types.

```
try {
    int[] a = {4, 8};
    System.out.println(a[3]);
    int n = Integer.parseInt("abc");
} catch (ArrayIndexOutOfBoundsException ex) {
    System.out.println("Invalid array position");
} catch (NumberFormatException ex) {
    System.out.println("Enter a valid number");
} catch (Exception ex) {
    System.out.println("Unexpected problem");
}
```

Why this order? A broad catch such as Exception placed first would make later specific catches unreachable.

Nested try-catch

A try block inside another try block is called a nested try. It is useful when an inner operation needs local handling while an outer block handles a broader operation.

```
try {
    System.out.println("Outer try");
    try {
        int x = 10 / 0;
    } catch (ArithmeticException ex) {
        System.out.println("Handled inside");
    }
    int[] a = new int[2];
    a[5] = 1;
} catch (ArrayIndexOutOfBoundsException ex) {
    System.out.println("Handled by outer catch");
}
```

- *The nearest matching catch is selected first.*
- *If the inner block does not handle an exception, it can propagate to the outer block.*
- *Keep nesting shallow where possible; helper methods often make code clearer.*

5. throw and throws

throw explicitly throws one exception object. throws appears in a method declaration and lists exception types that may pass to the caller.

```
// throw: create and throw an exception
static void validateAge(int age) {
    if (age < 18) {
        throw new IllegalArgumentException("Age must be 18+");
    }
}

// throws: declare a possible checked exception
static String readData() throws IOException {
    return java.nio.file.Files.readString(
        java.nio.file.Path.of("data.txt"));
}
```

Difference

throw: used inside a method/block; followed by an exception object; actually transfers control.
throws: used in method/constructor signature; followed by exception class names; informs callers of possible exceptions.

Calling a method that declares throws

```
try {
    String text = readData();
    System.out.println(text);
} catch (IOException ex) {
    System.out.println("Could not read file: " + ex.getMessage());
}
```

6. Exception propagation and custom exceptions

Propagation

When a method does not catch an exception, the exception travels back through the call stack to its caller. This is called exception propagation. If no matching handler exists, the thread terminates after the uncaught exception is reported.

main() calls service()
↓
service() calls repository()
↓
repository() throws IOException
↓
repository has no handler → returns exception to service
↓
service has no handler → returns exception to main
↓
main catches it → handled

```
static void third() {
    int n = 5 / 0; // throws ArithmeticException
}
static void second() { third(); }
static void first() { second(); }

public static void main(String[] args) {
    try {
        first();
    } catch (ArithmeticException ex) {
        System.out.println("Caught in main");
    }
}
```

Custom exception

Create a custom exception when a domain-specific failure deserves a meaningful name, such as InsufficientBalanceException or InvalidOrderException.

```
class InsufficientBalanceException extends Exception {
    InsufficientBalanceException(String message) {
        super(message);
    }
}

class BankAccount {
    void withdraw(double balance, double amount)
            throws InsufficientBalanceException {
        if (amount > balance) {
            throw new InsufficientBalanceException(
                "Balance is not sufficient");
        }
    }
}
```

Extend Exception for a checked custom exception, or RuntimeException for an unchecked one. Include a useful message and preserve a cause when wrapping another exception.

7. try-with-resources

Try-with-resources automatically closes resources that implement AutoCloseable. It is preferred for files, streams, readers, database resources and similar objects.

```
import java.io.*;

class ResourceDemo {
    public static void main(String[] args) {
        try (BufferedReader br = new BufferedReader(
                new FileReader("data.txt"))) {
            System.out.println(br.readLine());
        } catch (IOException ex) {
            System.out.println("File problem: " + ex.getMessage());
        }
    }
}
```

- *Resources are declared in parentheses after try.*
- *Resources are closed automatically when the block exits, even if an exception occurs.*
- *Multiple resources can be separated with semicolons; they close in reverse order of declaration.*
- *A resource must implement AutoCloseable (or Closeable).*

Traditional vs try-with-resources

```
// Traditional approach: easy to forget cleanup
FileInputStream in = null;
try {
    in = new FileInputStream("input.txt");
    // read data
} finally {
    if (in != null) in.close();
}

// Preferred: automatic cleanup
try (FileInputStream in =
         new FileInputStream("input.txt")) {
    // read data
}
```

8. Multi-catch and suppressed exceptions

Multi-catch

Use one catch block for unrelated exception types that need the same handling. Separate types with the pipe symbol |.

```
try {
    // operations that may fail in either way
} catch (IOException | NumberFormatException ex) {
    System.out.println("Input problem: " + ex.getMessage());
}
```

The alternatives in a multi-catch must not be related by inheritance. The catch parameter is effectively final, so do not assign a new value to it.

Suppressed exceptions

If the try-with-resources body throws an exception and closing a resource also throws, the body exception remains the primary exception. The close failure is attached as a suppressed exception.

```
try (MyResource r = new MyResource()) {
    throw new IOException("Main operation failed");
} catch (IOException ex) {
    System.out.println(ex.getMessage());
    for (Throwable s : ex.getSuppressed()) {
        System.out.println("Close failure: " + s.getMessage());
    }
}
```

Use getSuppressed() to inspect suppressed exceptions. This mechanism prevents a cleanup failure from silently replacing the original failure.

9. assert statement and assertion messages

An assert statement checks an assumption during development. If the condition is false while assertions are enabled, Java throws AssertionError.

```
int age = -2;
assert age >= 0;
assert age >= 0 : "Age cannot be negative";
```

Enable assertions

```
// Run the class with assertions enabled:
java -ea MyProgram

// Disable assertions (default in ordinary runs):
java MyProgram
```

• Assertions are disabled by default unless enabled with -ea (or a suitable package/class option).

• The optional message after ':' explains the failed assumption.

• Do not use assertions to validate public method arguments or user input; assertions may be disabled.

• Do not put essential program actions inside an assert expression, because the expression may not execute.

Assertion flow

Evaluate assertion condition
↓
True → continue program
False?
↙ ↘
Assertions enabled Assertions disabled
↓ ↓
Throw AssertionError Assertion is skipped

10. Assertions vs exception handling

Assertions and exceptions both detect problems, but they serve different purposes.

Assertions	*Exception handling*
Check internal assumptions / invariants.	*Handle expected or possible runtime failures.*
Can be disabled at runtime.	*Exception handling is part of normal program execution.*
Failure produces AssertionError.	*Failure produces a specific exception object.*
Useful during development and testing.	*Useful in production for recovery, reporting or cleanup.*
Not for validating external input.	*Use for invalid input, file, network and business-operation failures.*

Example

```
// Exception: validate data received from a user
if (amount <= 0) {
    throw new IllegalArgumentException("Amount must be positive");
}

// Assertion: verify an internal invariant
assert total >= 0 : "Internal total became negative";
```

Rule: Use exceptions for conditions callers must handle. Use assertions to document assumptions that should never be false if the program is correct.

11. Exception-handling best practices

• *Catch the most specific exception type you can handle.*

• *Do not use an empty catch block; log, recover, rethrow or explain why ignoring is safe.*

• *Never use exceptions for ordinary control flow.*

• *Preserve the original cause when wrapping: new MyException("message", cause).*

• *Give clear, actionable messages; avoid leaking passwords, tokens or sensitive data into logs.*

• *Use try-with-resources for every closeable resource.*

• *Do not catch Error or Throwable in routine application code.*

• *Do not log an exception and then rethrow it at every layer; this creates duplicate logs.*

• *Handle an exception at the layer that has enough context to recover or communicate meaningfully.*

• *Document checked exceptions that callers need to know about.*

• *Keep finally blocks simple; avoid return or throw from finally.*

• *Validate external input explicitly; do not rely on assert.*

Good vs poor handling

```
// Poor: hides the problem
try {
    saveFile();
} catch (Exception ex) {
}

// Better: handle with context
try {
    saveFile();
} catch (IOException ex) {
    System.err.println("Could not save file: " + ex.getMessage());
    throw ex; // if caller should decide recovery
}
```

A practical decision flow

Something can fail?
↓
Can this layer recover meaningfully?
↙ ↘
Yes No
↓ ↓
Catch specific type Declare / propagate it
↓ ↓
Recover or report Caller with context handles it
↓
Close resources automatically with try-with-resources

12. Quick revision and practice

One-minute revision

- *Throwable is the root of Java's Error and Exception branches.*
- *Checked exceptions: compiler requires catch or throws.*
- *Unchecked exceptions: RuntimeException and Error families.*
- *try contains risky code; catch handles; finally is for cleanup.*
- *throw throws an object; throws declares possible exception types.*
- *Propagation moves an uncaught exception up the call stack.*
- *Custom exceptions model meaningful application-specific failures.*
- *try-with-resources closes AutoCloseable resources.*
- *Multi-catch shares handling; suppressed exceptions preserve close failures.*
- *assert checks internal assumptions and is disabled by default.*

Practice questions

1. *Draw the Throwable hierarchy and give two examples each of Error and Exception.*
2. *Explain checked and unchecked exceptions with code examples.*
3. *What is the difference between throw and throws?*
4. *Explain exception propagation with a method-call diagram.*
5. *Create a checked custom exception named InvalidAgeException.*
6. *Why is try-with-resources safer than manual closing?*
7. *What is a suppressed exception? How can it be retrieved?*
8. *When should assertions not be used?*
9. *Why must specific catch blocks appear before general catch blocks?*
10. *List five exception-handling best practices.*

Mini coding task

```
Create a method transfer(balance, amount).
• Reject amount <= 0 with IllegalArgumentException.
• Throw a custom InsufficientBalanceException if amount > balance.
• Catch the custom exception in main and print its message.
• Add an assertion that the final balance is non-negative.
```

Exam tip: In every answer, define the concept, state the syntax/rule, show a short Java example, and mention one practical use.